Fresh-Model Training Results — Demand Forecasting

Name: Adam Aberbach
Date: 2026-04-14
Supervisor: Mohammed Reda

Executive Summary

We retrained the demand forecasting pipeline from scratch using a **disciplined train / val / test split** and found a configuration that delivers a **31% cost reduction** vs the production baseline, deployable today without any architectural changes.

Recommended production config:

Model	Features	Training	Test cost	vs Baseline
XGBoost-GPU, per-group buffers, cap=3.0	27 (lag/roll + shelf)	3 seconds on A100	**6,558,810 DKK**	**−31.0%**

Baseline (flat buf=1.40): 9,498,367 DKK. Absolute saving: **2,939,557 DKK over 14 test days**.

What We Tested

Every experiment below uses the same 14-day held-out test window, same shelf-aware cost function (`waste + 1.5 × stockout`, waste fraction scaled by perishability), and buffers tuned on a 14-day validation split **before** the test (no leakage).

Models trained fresh

#	Model	Training time	Test MAE	Best test cost (DKK)	vs buf=1.40
1	LightGBM (CPU, production default)	67s	1.78	~8,419,057	−11%
2	**XGBoost-GPU**	**3.1s**	**1.46**	**5,906,055** (unconstrained) / **6,558,810** (cap=3)	**−37.8% / −31.0%**
3	CatBoost-GPU	8.3s	1.67	9,067,154	marginal
4	Blend (LGB+XGB+CB)	94s	1.73	8,584,654	−10%
5	SoftProbV2 (per-store LGB)	337s	1.74	8,500,783	−10%
6	MLP (PyTorch, 27 features)	6.5s	1.23	5,938,695 (unc) / 7,615,921 (cap=3)	ties XGB unc, worse at cap

Winner: XGBoost on GPU. 22× faster than CPU LightGBM, better predictions, better cost.

Neural time-series models (darts library, 1500 high-volume subset)

For full due-diligence, we also ran the main neural time-series architectures on a 1500-series subset (top by volume), to confirm they don't beat XGB. Same train/val/test split, same cost function. XGB re-evaluated on same subset for apples-to-apples.

Without covariates (10 epochs, default setup):

Model	Test MAE	Best cost (1500 subset)	Train time	vs XGB subset
XGB per-group cap=3	7.02	**2,200,459 DKK**	**1.5 s**	—
DeepAR (2-layer LSTM, 64 hidden)	7.72	2,494,011	29.8 s	+13.3%
N-HiTS (3 stacks, 2 layers, 256 wide)	6.87	2,725,403	39.9 s	+23.9%
TFT (Temporal Fusion Transformer)	8.84	6,279,909	178.7 s	+185% (worst)

With covariates (20 epochs, static + future calendar features):

Model	Test MAE	Best cost (1500 subset)	Train time	vs XGB subset	Δ vs no-cov
XGB per-group cap=3	7.02	**2,200,459**	**1.5 s**	—	—
DeepAR + future covs (LSTM 96, 2 layers)	8.30	3,025,312	84 s	+37.5%	+21% worse
N-HiTS (4 stacks, 3 layers, 512 wide, 20 ep)	6.99	3,135,556	169 s	+42.5%	+15% worse
TFT + static + future covs (64 hidden, 20 ep)	8.74	4,059,438	821 s	+84.5%	**+35% better**

Key findings from the covariates experiment: 1. **TFT improved by 35%** when given proper covariates — confirming the architecture does benefit from side information. But it still loses to XGB by 85%. 2. **DeepAR and N-HiTS got worse with more epochs** — overfit the training data. The 10-epoch version was actually closer to XGB. 3. **XGB's 1.5 s training beats 14-minute TFT** on the same subset, same features. Not a close call.

Why XGB wins even with full feature parity: - Tree splits handle the asymmetric cost (waste cheap + stockout expensive) naturally; neural MSE loss fights against it. - Per-(store, item) lag features are extremely predictive; tree models consume them without any embedding trickery. - XGB has 20+ years of library optimization; darts neural models are newer and less mature on retail data.

Shelf-life features

Adding 11 shelf-life features (`shelf_life_days`, `perishability`, `storage_type`, `effective_waste_frac`, interactions with rolling means, etc.) at buf=1.40 alone did **not** reduce cost meaningfully — it actually increased it by 0.04–1.24% across models.

The value of shelf features shows up **only when buffers are tuned per shelf group**. The unconstrained optimizer assigns each group its optimal buffer:

Shelf group	Best buffer (unconstrained)	Best buffer (cap=3)
ultra_fresh (≤3 days)	1.98	1.98
fresh (4–14 days)	3.19	3.00
medium (15–90 days)	6.35	3.00 (saturates)
long_shelf (91–365 days)	10.00 (saturates)	3.00 (saturates)
non_perishable (>365 days)	10.00 (saturates)	3.00 (saturates)

Why the saturation? For non-perishable items, waste cost is ≈ 0 (they don't expire), so the optimizer says "stock as much as possible to eliminate stockouts." Capping at 3× is a pragmatic choice: **cap=3 captures 82% of the theoretical win at a realistic warehouse footprint.**

Buffer cap sweep (production deployment options)

Every 0.5× of additional cap buys diminishing returns:

Cap	Test cost (DKK)	vs buf=1.40	Notes
2.0	7,646,258	−19.5%	very conservative
2.5	6,918,902	−27.2%	
3.0	**6,558,810**	**−31.0%**	**sweet spot**
4.0	6,206,702	−34.7%	
5.0	6,053,516	−36.3%	
10.0	5,906,055	−37.8%	theoretical ceiling

Things that did NOT help

1. **Per-store buffer tuning** (tune one buffer per store on val → apply to test): **5,891,600 DKK** at cap=10. Only 14K DKK better than global per-group — not worth operational complexity.
2. **Store metadata features** (`chain_id`, `area_id`, `type_id`, `primary_cuisine_id`, `seasonal`, `store_age_days`, `hours_per_week`, `days_open_per_week`, `weekend_open`, `closed_monday`): **5,924,010 DKK** — slightly **worse** than the lean feature set. XGB already captures store behavior through per-(store, item) lag features.
3. **Per-store × per-group buffers** (up to 2,470 buffers total): **5,891,600 DKK** — same as above, no material gain.
4. **Per-item buffers for top-100 high-error items**: **5,947,325 DKK** — worse than per-group. Overfits validation noise.

Recommended Deployment Recipe

```
# 1. Train the model (once per week or when drift detected)
model = xgb.XGBRegressor(
    n_estimators=400, learning_rate=0.05, max_depth=7,
    min_child_weight=5, subsample=0.9, colsample_bytree=0.8,
    reg_alpha=0.1, reg_lambda=1.0,
    tree_method="hist", device="cuda",
    random_state=42, n_jobs=-1,
)
model.fit(X_train, y_train, sample_weight=decay_weights, verbose=False)
# Runtime: ~3 seconds on A100

# 2. Freeze the per-group buffer table (tune on val, deploy once)
GROUP_BUFFERS = {
    "ultra_fresh":    1.98,
```

```
    "fresh":          3.00,
    "medium":         3.00,
    "long_shelf":     3.00,
    "non_perishable": 3.00,
}

# 3. For a new (store, item, date) recommendation:
def recommend_quantity(store_id, item_id, date):
    features = build_feature_row(store_id, item_id, date)  # 27 features
    base = max(0, model.predict(features)[0])
    group = classify_shelf(item_id)           # look up in SHELF_LOOKUP
    return int(np.ceil(base * GROUP_BUFFERS[group]))
```

Features (27):
`dow, is_weekend, dom, month, lag_{1,2,3,7,14,28}, rmean_{3,7,14,28}, rstd_{3,7,14,28}, sl, ag, per, storage_type, effective_waste, logsl, turn, shelf_x_rolling7, perishable_x_weekend`

Retraining cadence: weekly on rolling 155-day window. Exponential decay with half-life = 6 days gives recent data higher weight.

Validation discipline: always tune buffers on a held-out val set (never the test). Walk-forward retune weekly.

Why XGBoost Wins

1. **Asymmetric-cost-friendly.** Tree splits can learn "when demand is high, predict high" without being punished by squared-error regularization. MLP with the same features and lower MAE actually produced *worse* cost, because MSE training fights the asymmetric waste-vs-stockout tradeoff.
2. **No tuning drama.** One set of hyperparameters, one training run, done in 3 seconds. Compare to the 32-minute Chronos fine-tune that made things worse (see Yahya's report).
3. **GPU acceleration is real.** A100 XGB takes 3 seconds for 2.2M rows × 27 features. CPU LightGBM takes 67 seconds for the same. That 22× speedup enables near-real-time retraining.
4. **Features carry store identity implicitly.** Per-(store, item) lag and rolling features already encode store behavior. Adding explicit store metadata was net-negative because of the extra parameters.

Risks & Caveats

1. **Cap=3 loses 652K DKK vs unconstrained.** If your warehouse can actually hold 5–10× predicted demand for shelf-stable items, cap=10 saves an extra 0.07% of annual sales. Low priority.
2. **6-month training window, one winter, no summer.** We can't learn year-over-year seasonality from this data. Plan to revisit buffers and features after a full annual cycle.
3. **Cold-start items** (new items with <28 days history) will have weak lag features. Fall back to a manual buffer (e.g., 1.5× manager's gut estimate) for the first two weeks of a new item.
4. **Test set is 14 consecutive days** (2026-03-21 → 2026-04-03). A single-window estimate. Walk-forward validation at 3–5 cutoffs would confirm robustness; recommended before full rollout.
5. **Our cost function is a model.** `waste + 1.5 × stockout` with a perishability-scaled waste fraction captures the main trade-off, but actual P&L has terms we're not modeling (supplier lead time, bulk-order discounts, return policies). Treat cost numbers as directional, not accounting-grade.

Next Steps

1. **Deploy XGB + cap=3 as the new baseline.** Roll out on 10% of stores (A/B against current flat=1.40), compare 14-day cost.
2. **Cold-start policy**: explicit fallback rules for items with <28 days history.
3. **Walk-forward validation** at 3 historical cutoffs to confirm the -31% number is robust.
4. **Retraining pipeline**: daily/weekly retrain job, frozen buffers, feature-drift monitoring.
5. **Cost-function review** with ops: confirm 1.5× stockout multiplier reflects actual business pain, adjust waste-fraction curve if needed.

Appendix: Feature Importance (XGBoost cap=3 model)

Rank	Feature	Importance
1	`rmean_7` (7-day rolling mean)	29.5%
2	`rmean_14`	11.4%
3	`rmean_28`	9.9%
4	`lag_14`	6.2%
5	`lag_7`	4.7%
6	`logsl` (log shelf life)	2.7%
7	`lag_28`	2.7%
8	`dow` (day of week)	1.8%
9	`month`	1.6%
10	`closed_monday` (store opens Monday?)	1.6%

Rolling demand features dominate. Shelf life enters at rank 6. Store-metadata features (other than `closed_monday`) all below 1% — confirms the "store identity is in the lags" hypothesis.